

Manual de Utilizacao - Pessoa RJ

1. Visao geral

Aplicacao desktop que centraliza o cadastro de pessoas fisicas no Consinco, executa consultas multi banco Oracle e dispara rotinas auxiliares de comprovantes.

2. Pre-requisitos

- Python 3.11+ com dependencias do projeto instaladas (pandas, requests, oracledb, tkinter, pillow, fpdf, openpyxl).
- Oracle Instant Client configurado quando o driver oracledb opera em modo thick.
- Credenciais validas para os bancos BNZ, MLT e ALI e para a API Totvs Consinco.

3. Estrutura principal

O pacote pessoa_rj contem configuracao, acesso a banco, cliente de API, utilitarios e interface Tk. SQLs e imagens residem em pessoa_rj/queries e pessoa_rj/assets respectivamente.

4. Executando a aplicacao principal

Modo desenvolvimento: `python -m pessoa_rj`.

Executavel: utilize o arquivo gerado no diretorio dist apos o empacotamento PyInstaller.

5. Fluxo de cadastro de pessoas

- 1) Fazer login na janela inicial usando usuario e senha Totvs.
- 2) Importar planilha XLSX com dados de pessoas ou buscar CPFs via bancos.
- 3) Filtrar e selecionar registros na grade principal.
- 4) Acionar Cadastrar Selecionados para consultar existencia e enviar novos cadastros.
- 5) Conferir o log em tela e o Excel exportado com resultado parcial.

6. Consulta multi banco

A janela Buscar Dados Bancos aceita CPFs livres ou importados de XLSX. As consultas sao executadas para os tres Oracle configurados e retornam DataFrame consolidado, pronto para integracao com a tela principal.

7. Integracao com API Totvs

Tokens sao obtidos via login e renovados automaticamente. O refresh token fica salvo em token_storage.json localizado no diretorio de execucao.

8. Ferramentas auxiliares

Extrair Comprovantes (convert.py) divide PDF macro usando relatorio TXT e gera ZIP/Excel de resumo.
Gerar Comprovante Trabalhista (comprov.py) monta recibos a partir de planilhas de base RJ e movimentacao C5.

9. Logs e arquivos de saida

Log principal: pessoa_rj.log no diretorio de execucao.

Excel de resultados: resultado_<nome_da_planilha>.xlsx salvo ao lado da planilha importada.
Arquivos auxiliares (PDF, ZIP, xlsx) são salvos na pasta escolhida pelo usuario em cada rotina.

10. Empacotamento

Utilize o comando PyInstaller documentado, incluindo assets, SQLs, icon e hidden imports:

```
pyinstaller --onefile --noconsole --icon pessoa_rj/assets/financeiro.ico --add-data  
pessoa_rj/assets/rodape.png;pessoa_rj/assets --add-data pessoa_rj/assets/logo.png;pessoa_rj/assets  
--add-data pessoa_rj/assets/financeiro.ico;pessoa_rj/assets --add-data  
pessoa_rj/queries/bnz.sql;pessoa_rj/queries --add-data pessoa_rj/queries/mlt.sql;pessoa_rj/queries  
--add-data pessoa_rj/queries/ali.sql;pessoa_rj/queries --hidden-import convert --hidden-import comprov  
--collect-all cryptography --collect-all oracledb --hidden-import cryptography.hazmat.primitives.kdf  
--hidden-import cryptography.hazmat.bindings._rust cadastro_pessoa_gui.py
```

11. Resolucao de problemas rapida

- Token expirado: remover token_storage.json e efetuar novo login.
- Falha Oracle: validar credenciais, rede VPN e variavel de ambiente do Instant Client.
- Assets ausentes no executavel: confirmar add-data no comando PyInstaller.